

Deep Q-Networks (DQN)

A Lecture on Deep Reinforcement Learning

Why DL? · Q-Learning · Architecture · Components · Flow · Variants

Predator-Prey Archetype Gridworld · 2026

Lecture Outline

1. Why Deep Learning?

Tabular limits & function approximation

2. RL Fundamentals

MDP, returns, value functions, Bellman

3. TD Learning & Q-Learning

TD updates, off-policy control

4. The DQN Insight

Scaling RL to high-dim spaces

5. DQN Architecture

CNN + FC network design

6. DQN Components

Replay, target net, ϵ -greedy, loss

7. End-to-End Flow

Training algorithm & pipeline

8. Example: CartPole

Worked example

9. DQN Variants

DDQN, Dueling, PER, Noisy, Rainbow

10. Key Takeaways

Summary & further reading

The Limits of Tabular RL

A Q-table stores one value per (state, action) pair.

- Works for small discrete worlds: Gridworld, FrozenLake, ...
- Exact lookup — convergence guaranteed

Q(s,a)	←	→	↑	↓
s □	0.12	0.85	0.23	0.07
s □	0.44	0.31	0.62	0.55
s □	0.08	0.19	0.91	0.03
s □	0.73	0.65	0.44	0.88
□	...	...	...	...

□ The Problem

Most real problems have enormous or continuous state spaces.

Atari: $\approx 10^{70000}$ states

A Q-table cannot scale.

The Curse of Dimensionality

State space grows exponentially with problem complexity.

Simple Grid

$4 \times 4 = 16$ states

Atari Pixels

$\approx 10^{70,000}$ states

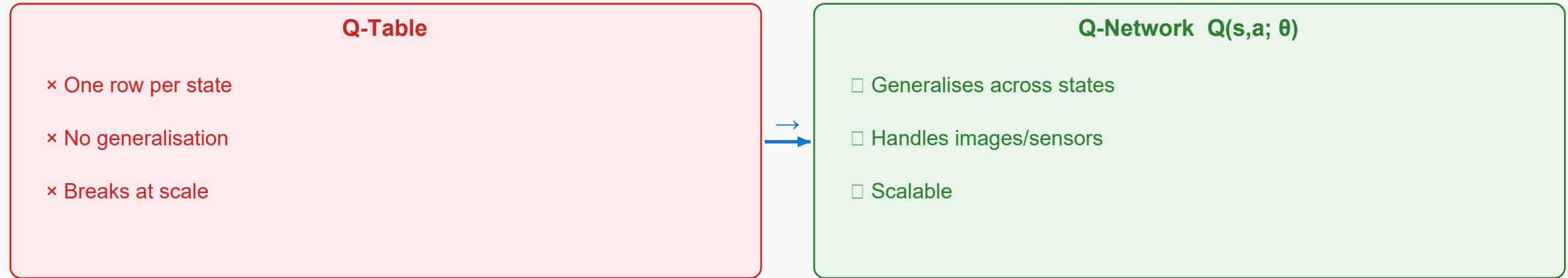
Continuous

∞ states

□ **Key insight:** We need to generalise across states — not memorise each one.

Deep Learning as Function Approximator

Replace the Q-table with a neural network that generalises.



$$Q^*(s, a) \approx Q(s, a; \theta) \quad \text{where } \theta \text{ are the network weights}$$

The network takes state s and outputs Q -values for every action a

The RL Framework: Markov Decision Process



MDP tuple: (S, A, P, R, γ) — States, Actions, Transitions, Rewards, Discount

Return & the Discount Factor γ

The agent maximises the expected discounted cumulative reward:

$$G_t = r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \dots = \sum_{k=0}^{\infty} \gamma^k r_{t+k}$$

G_t = return from timestep t $\gamma \in [0, 1)$ = discount factor

$\gamma = 0$

Myopic — only
next reward matters

$\gamma \rightarrow 1$

Far-sighted — all
future rewards equal

$\gamma = 0.99$

Typical choice
(balances both)

Value Functions: $V(s)$ and $Q(s,a)$

Two key functions describe how good a state or action is:

State-Value $V(s)$

$$V^{\pi}(s) = E_{\pi} [G_{\infty} \mid s_{\infty} = s]$$

Expected return from state s under policy π

"How good is it to be in state s ?"

Used in policy evaluation

Action-Value $Q(s,a)$

$$Q^{\pi}(s,a) = E_{\pi} [G_{\infty} \mid s_{\infty} = s, a_{\infty} = a]$$

Expected return taking action a in state s , then following π

"How good is action a in state s ?"

DQN approximates $Q^*(s,a)$ directly!

Optimal policy: $\pi^*(s) = \operatorname{argmax}_a Q^*(s,a)$ — just pick the highest Q -value

The Bellman Optimality Equation

The optimal Q-function satisfies a recursive self-consistency equation:

$$Q^*(s, a) = E [r + \gamma \cdot \max_{a'} Q^*(s', a') \mid s, a]$$

Value of (s,a) = immediate reward + discounted best achievable value from next state

Intuition:

- $Q^*(s,a)$ is defined in terms of itself — a recursive equation
- Solved iteratively by repeated application (Q-learning, Value Iteration)
- DQN trains a neural network to satisfy this equation

- Q-Learning solves this iteratively with tabular updates.
- DQN approximates Q^* with a neural network trained by gradient descent.

Temporal Difference (TD) Learning

TD learning updates value estimates online — no episode end needed.

$$V(s_t) \leftarrow V(s_t) + \alpha \cdot \delta_t$$

where $\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t)$ (TD error)

α = step size / learning rate

Monte Carlo

- Wait until episode ends
- Use full return G_t
- High variance, unbiased

TD Learning

- Update after each step
- Bootstrap from $V(s_{t+1})$
- Lower variance** $\square$

Q-Learning: Off-Policy TD Control

Q-learning applies TD directly to learn $Q^*(s,a)$ without a model.

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha [r_t + \gamma \max_a Q(s_{t+1}, a) - Q(s_t, a_t)]$$

α = learning rate $\max_a$ = greedy selector γ = discount

Key properties:

- Off-policy: learns Q^* regardless of which policy collects data
- Model-free: doesn't need to know $P(s'|s,a)$ or $R(s,a)$
- Provably converges to Q^* given sufficient exploration + decaying α

- **Q-Learning is the direct predecessor of DQN.**

DQN replaces the Q-table with a deep neural network.

Why Q-Learning Fails at Scale

Naively adding a neural network to Q-learning breaks training.

Problem 1 · Correlated data

Consecutive (s,a,r,s') are highly correlated.
Neural nets need i.i.d. samples $\rightarrow$ divergence.

Problem 2 · Moving targets

Target $r + \gamma \max Q(s'; \theta)$ shifts every update.
Chasing a moving target $\rightarrow$ instability.

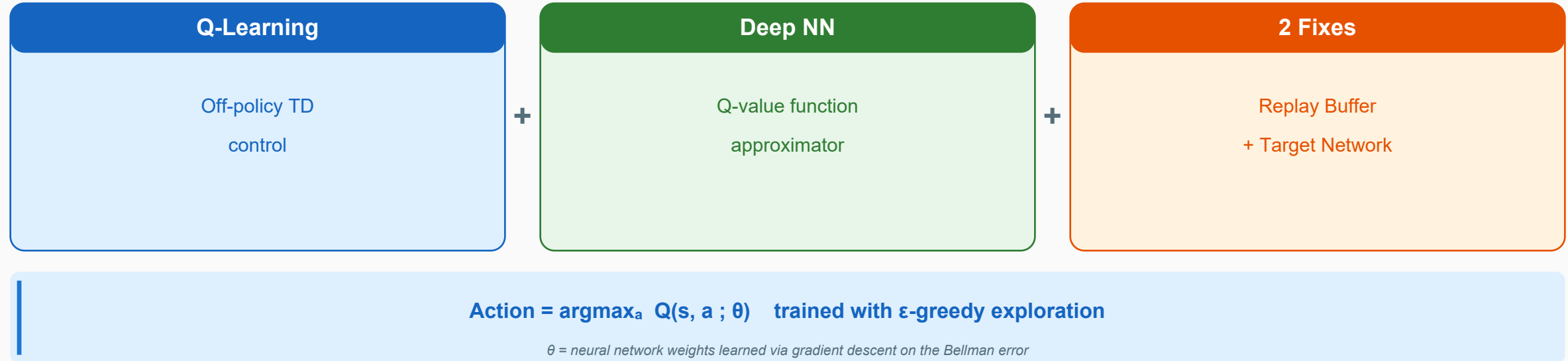
Problem 3 · Catastrophic forgetting

Updates on new data overwrite old knowledge.
Net forgets what it learned earlier.

- DQN solves Problems 1 & 2 with Experience Replay + Target Network.

DQN: The Big Idea

DQN = Q-Learning + Deep Neural Network + two stabilising tricks.



The Atari Breakthrough (Mnih et al., Nature 2015)

DeepMind trained one DQN agent on 49 Atari 2600 games.

Input: Raw 84×84 greyscale pixels — no hand-crafted features

Output: Q-value per joystick action

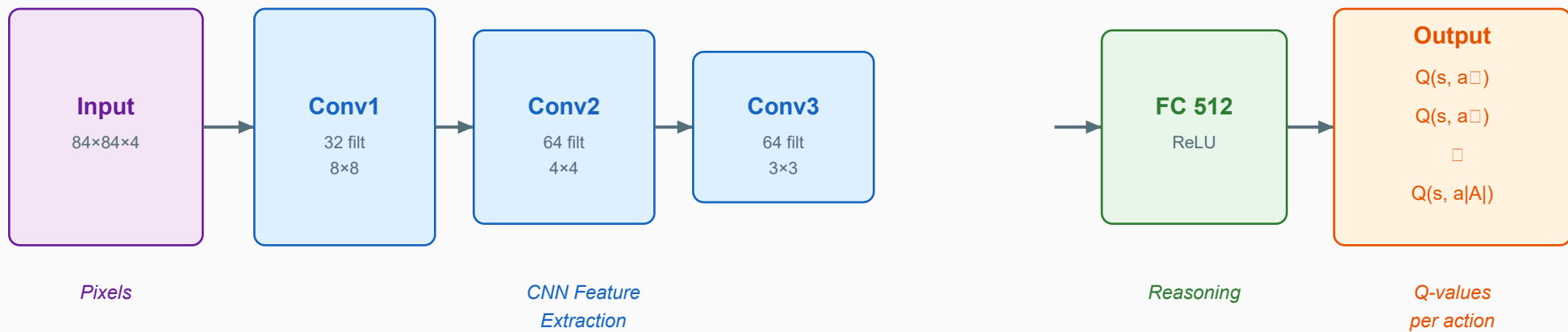
Training: Same hyperparameters for every game

Result: Superhuman on 29 / 49 games

Published: Nature, February 2015 — landmark paper for deep RL

- ❑ One algorithm, raw pixels, superhuman performance on diverse games.
Demonstrated that deep RL can generalise end-to-end across tasks.

DQN Network Architecture



Input Preprocessing (Atari)

Raw Atari frames are preprocessed before entering the CNN.

1. Greyscale

210×160 RGB → 210×160 greyscale (channels: 3 → 1)

2. Crop & Resize

Resize to 84×84 pixels (standardise spatial resolution)

3. Frame Stack

Stack 4 consecutive frames → 84×84×4 (captures motion & velocity)

4. Normalise

Scale pixel values to [0, 1] (stabilises gradient flow)

4 stacked frames give the network a sense of velocity and direction.
Without stacking the agent cannot tell if a ball is moving left or right.

The Two Core Problems DQN Solves

Adding a NN to Q-learning introduces two new training problems:

Problem 1 Correlated Data

Consecutive (s, a, r, s') are correlated.

NNs need i.i.d. data.

→ training diverges

Fix: Experience Replay

Problem 2 Moving Targets

Target $r + \gamma \max Q(s'; \theta)$ shifts

every gradient step.

→ training diverges

Fix: Target Network

□ DQN solves both with Experience Replay and a frozen Target Network.

Component 1: Experience Replay

Store all transitions in a buffer; train on random mini-batches.



Mini-batch sampled randomly (highlighted above)

- Breaks temporal correlations — training data is i.i.d.
- Each transition can be replayed many times (data efficiency)
- Stabilises the training distribution over time

Component 2: Target Network

Keep a frozen copy of the network to compute stable TD targets.



$$\text{Target } y_i = r_i + \gamma \cdot \max_a Q(s'_i, a; \theta_{\square})$$

$\theta_{\square}$ is frozen during C gradient steps — provides stable regression targets

Component 3: ϵ -Greedy Exploration

Balance exploration with exploitation during training.

$$\begin{aligned}\pi(s) &= \text{random action} && \text{with probability } \epsilon \\ \pi(s) &= \operatorname{argmax}_a Q(s, a; \theta) && \text{with probability } 1 - \epsilon\end{aligned}$$

ϵ anneals linearly: $\epsilon_{start} (1.0) \rightarrow \epsilon_{end} (0.1)$ over M training steps

Exploration $\epsilon: 1.0 \rightarrow 0.1$

Exploit

- ☐ High ϵ early: agent explores freely, populates the replay buffer
- ☐ ϵ decays over 1M frames in the original DQN paper
- ☐ Low ϵ later: agent exploits its learned policy

Component 4: The Loss Function

Minimise squared Bellman error using sampled mini-batches.

$$L(\theta) = E(s,a,r,s') \sim D [(y_i - Q(s, a; \theta))^2]$$

$$y_i = r + \gamma \cdot \max_{a'} Q(s', a'; \theta) \quad (\text{stop gradient!})$$

Gradient $\nabla_{\theta} L$ flows only through $Q(s,a;\theta)$ — NOT through y_i

In practice:

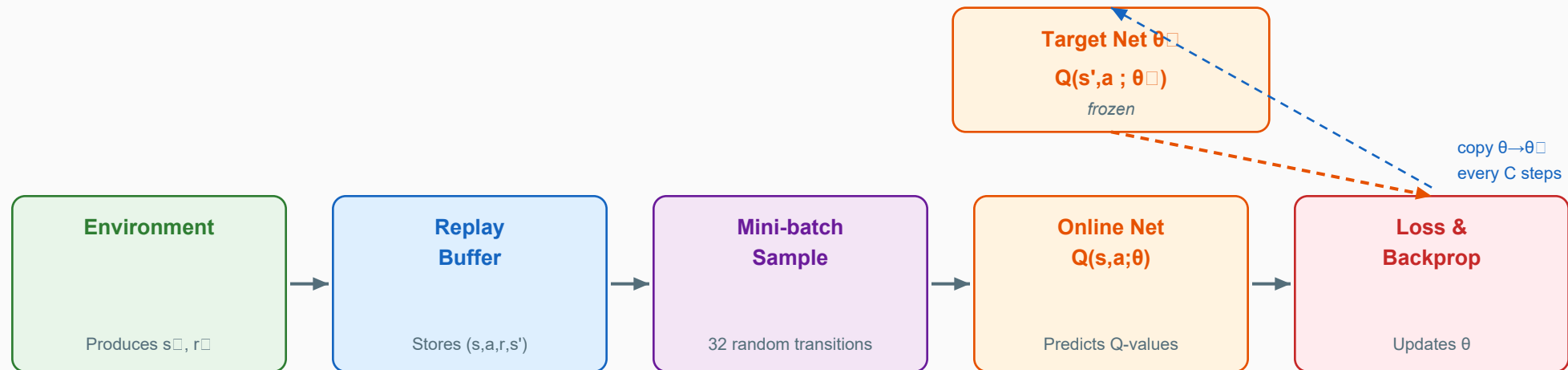
- ☐ Huber loss instead of MSE — more robust to large TD errors
- ☐ Clip rewards to $[-1, +1]$ to normalise across different games
- ☐ Update online θ every step; copy to θ_{old} every $C \approx 1000$ steps

- ☐ Never backpropagate through the target y_i — it must be treated as a constant.

DQN Training Algorithm

Initialise	replay buffer D (capacity N), online network $Q(\theta)$, target $\theta_{\square} \leftarrow \theta$
For episode	1 ... M:
Observe	initial state $s_{\square}$
For t	= 1 ... T:
Select	$a_{\square}$: random w.p. ϵ , else $\operatorname{argmax}_a Q(s_{\square}, a; \theta)$ [ϵ -greedy]
Execute	$a_{\square} \rightarrow$ observe $r_{\square}, s_{\square\square\square}$
Store	$(s_{\square}, a_{\square}, r_{\square}, s_{\square\square\square})$ in D
Sample	mini-batch $\{(s_i, a_i, r_i, s'_i)\}$ from D
Compute	$y_i = r_i + \gamma \max_a Q(s'_i, a; \theta_{\square})$ [terminal: $y_i = r_i$]
Step	gradient descent on $L(\theta) = \sum (y_i - Q(s_i, a_i; \theta))^2$
Every C	$\theta_{\square} \leftarrow \theta$

Data Flow: One Training Step



Putting It All Together

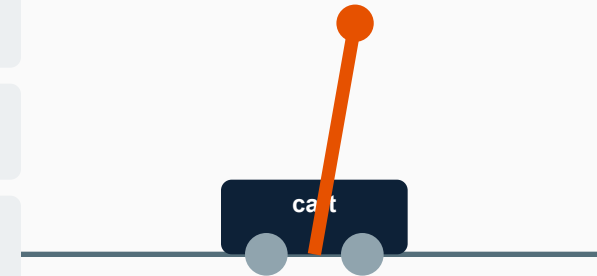
Each training step executes six operations in order:

- **Act** ϵ -greedy selects action a_i from online network $Q(s,a;\theta)$
- **Store** Transition (s_i, a_i, r_i, s_{i+1}) appended to replay buffer D
- **Sample** Random mini-batch of 32 transitions drawn from D
- **Target** Frozen θ_i computes $y_i = r + \gamma \max_{a'} Q(s'; \theta_i)$
- **Update** Gradient step minimises $(y_i - Q(s_i, a_i; \theta))^2 \rightarrow$ update θ
- **Sync** Every C steps: copy online weights $\theta_i \leftarrow \theta$

Example: CartPole-v1

Balance a pole on a moving cart — a classic RL benchmark.

State:	4D: [cart x , cart $\dot{x}$, pole θ , pole $\dot{\theta}$]
Actions:	2 discrete: push left / push right
Reward:	+1 for every step the pole stays upright
Goal:	Survive 500 steps (solved threshold)
Network:	4 inputs $\rightarrow$ FC(64, ReLU) $\rightarrow$ FC(64, ReLU) $\rightarrow$ 2 Q-values



DQN Applied to CartPole: Step by Step

One full training interaction traced through DQN:

Observe

$s = [0.02, -0.04, 0.01, 0.03]$

ϵ -greedy

$\epsilon=0.9 \rightarrow$ random action: push right ($a=1$)

Transition

$r=1.0$, $s'=[0.03, 0.15, 0.03, -0.26]$ (pole still upright)

Buffer

Store $(s, 1, 1.0, s')$ in D. Buffer reaches 32 $\rightarrow$ begin sampling

Target

$y = 1.0 + 0.99 \times \max Q(s'; \theta_{\square}) = 1.0 + 0.99 \times 0.54 \approx 1.53$

Online

$Q(s, 1; \theta) = 0.31$

Loss

$L = (1.53 - 0.31)^2 = 1.49 \rightarrow$ backprop $\rightarrow \theta$ updated

Repeat

Over 50k steps ϵ anneals $1.0 \rightarrow 0.1$; agent learns to balance

Double DQN (DDQN)

DQN overestimates Q-values — the same network selects and evaluates.

DQN target: $y_i = r_i + \gamma \max_a Q(s'_i, a; \theta^-)$ ← same net selects & evaluates

Problem: maximisation bias → systematic overestimation of Q-values

DDQN target: $y_i = r_i + \gamma Q(s'_i, \operatorname{argmax}_a Q(s'_i, a; \theta^-); \theta^-)$

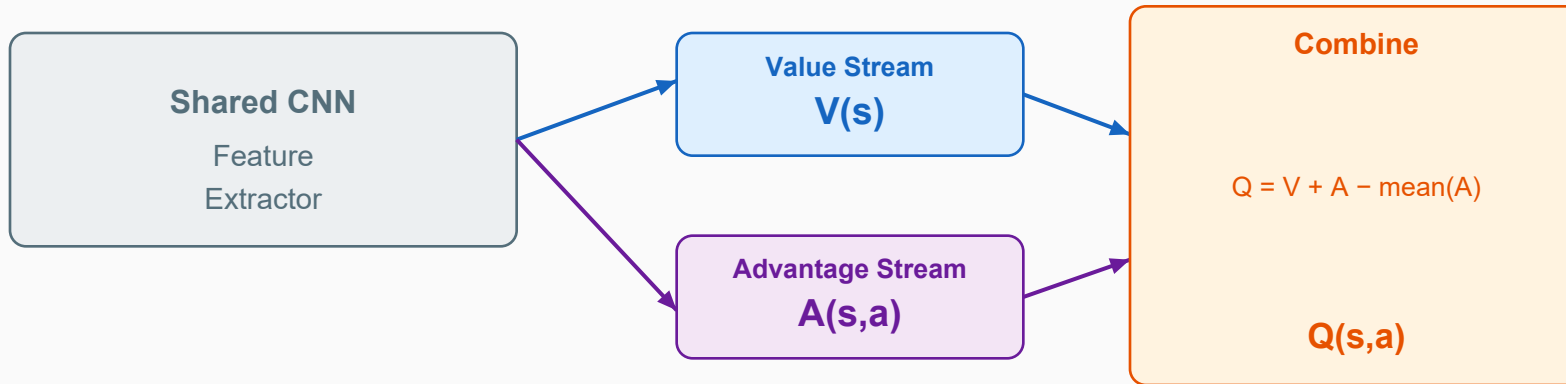
Online θ selects, target θ^- evaluates → decoupled, less biased

Key change:

- Online network θ : 'Which action looks best from s' '?
- Target network θ^- : 'What is that action actually worth'?
- Tiny code change — large improvement in value estimates

Dueling Network Architecture

Decompose $Q(s,a)$ into state value $V(s)$ and advantage $A(s,a)$.



Benefit: network learns $V(s)$ even for actions that don't matter → faster convergence.

Prioritised Replay · Noisy Nets · Rainbow

Prioritised Experience Replay (PER)

Sample high-TD-error transitions more

- Learn more from surprising experiences
- IS weights correct sampling bias

Noisy DQN

- Learnable noise in FC layer weights
- State-dependent exploration (smarter)
- Replaces ϵ -greedy schedule entirely

Rainbow (2017)

- Combines 6 DQN improvements:
- DDQN + Dueling + PER +
- Noisy + Distributional + n-step
- State of the art on Atari (at time)

Key Takeaways

DQN = Q-Learning + Deep NN + Experience Replay + Target Network

Experience Replay: random mini-batches break temporal correlations

Target Network: frozen copy stops the agent chasing a moving target

ϵ -greedy exploration anneals from 1.0 $\rightarrow$ 0.1 over training

Loss = Huber($(r + \gamma \max_{a'} Q(s'; \theta^-) - Q(s, a; \theta))^2$) with stop-gradient

DDQN, Dueling, PER, Rainbow each fix a specific remaining weakness